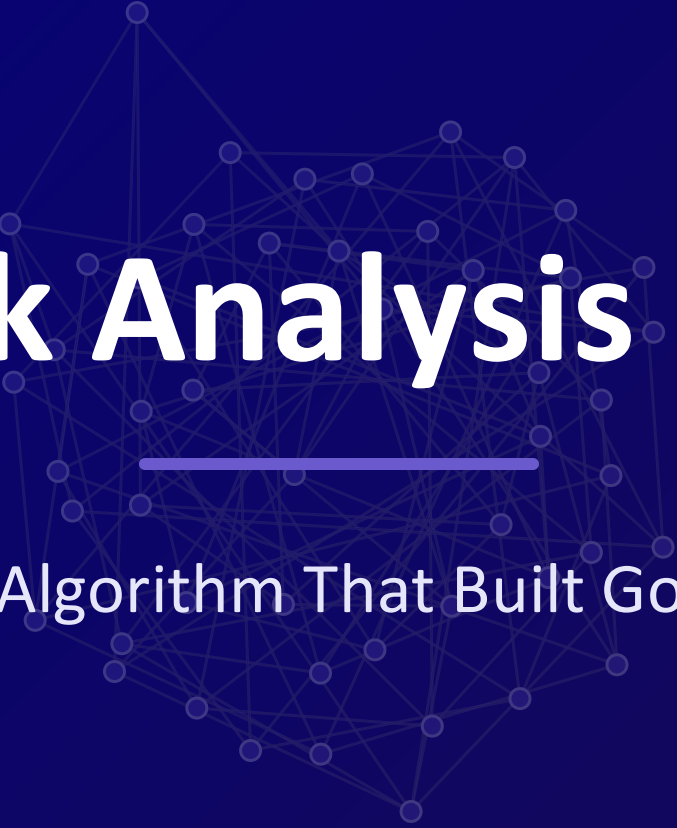




Week 7: Link Analysis & PageRank

The Algorithm That Built Google

Course Plan

Week 1 Introduction to IR	Week 2 Preprocessing & Text Analysis	Week 3 Inverted Indexes
Week 4 Index Construction	Week 5 Vector Space Model	Week 6 Evaluation Metrics
Week 7 Link Analysis & PageRank	Week 8 XML Retrieval	Week 9 Web Search Basics
Week 10 Text Classification	Week 11 Clustering	Week 12 Machine Learning for IR

 We Are Here

The Web as a Graph

We model the World Wide Web as a massive **directed graph** $G = (V, E)$.

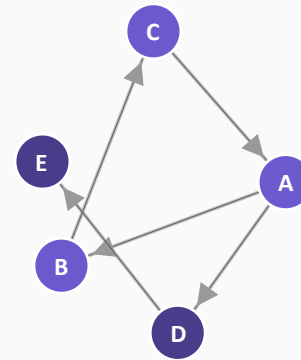
Nodes (V)

Represent **web pages** (HTML documents).

Edges (E)

Represent **hyperlinks**. An edge (u, v) exists if page u links to page v .

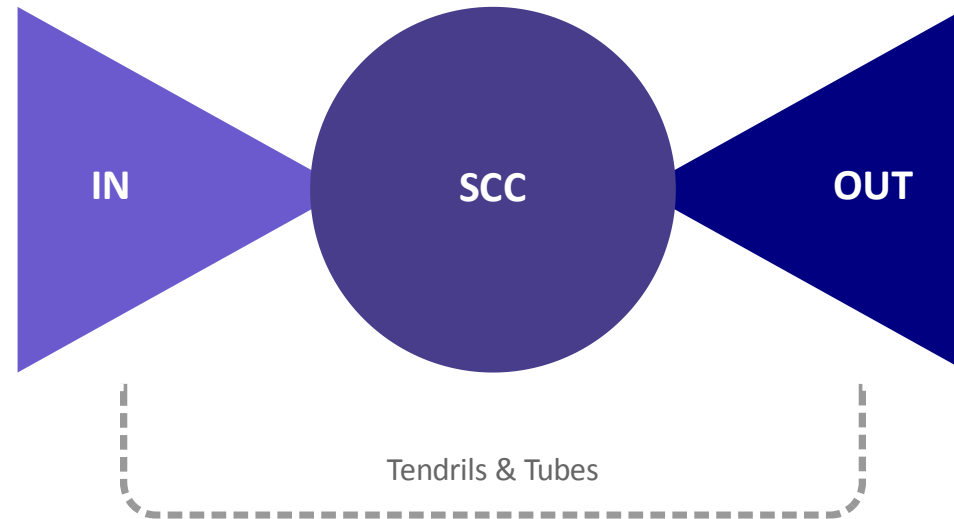
Key Property: Links are *directed*. A link is an endorsement from source to target.



A simple directed graph representing web structure.

The Bow-Tie Structure

Macroscopic structure of the Web (Broder et al., 2000)



~28%

Strongly Connected Component (SCC)
The "Core"

~22%

IN Component
Origination pages

~22%

OUT Component
Destination pages

Source: Broder et al., "Graph structure in the web", WWW 2000

Why Analyze Links?

Links as Votes

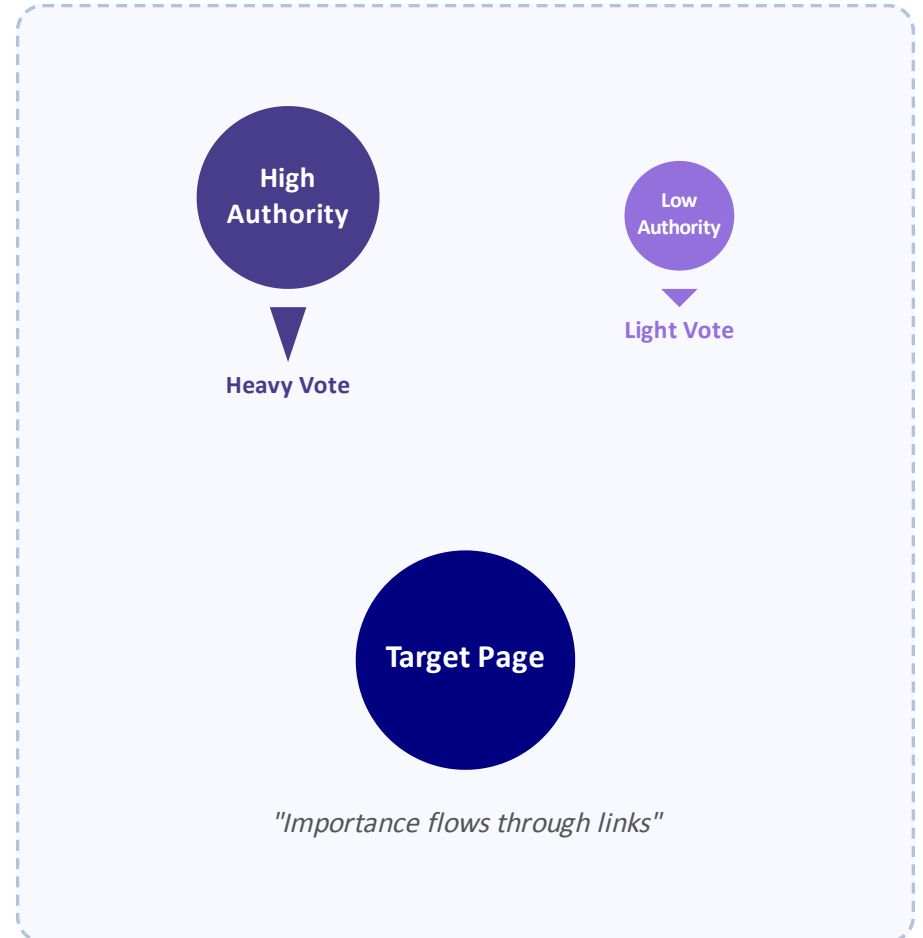
The fundamental hypothesis of link analysis: A hyperlink from Page A to Page B is an **endorsement** of Page B's content.

The Citation Analogy

Similar to academic citations. A paper cited by many other papers is likely authoritative and important in its field.

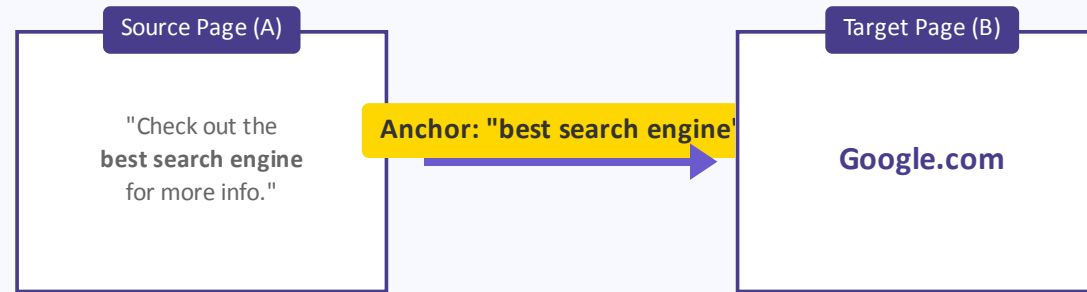
Not All Votes Are Equal

A link from a high-authority page (e.g., *The New York Times*) should count more than a link from a random personal blog. This leads to a **recursive definition** of importance.



Anchor Text: A Recap

Anchor text is the clickable text in a hyperlink. It serves as a **peer review** or annotation of the target page, often providing a better description than the page itself.



! Case Study: The "Google Bomb" (c. 2003)

The Exploit: Thousands of websites linked to George W. Bush's biography with the anchor text *"miserable failure"*.

The Result: Searching for "miserable failure" on Google ranked the President's bio as the **#1 result**, even though the phrase didn't appear on the page.

PageRank: The Intuition

1. Links as Votes

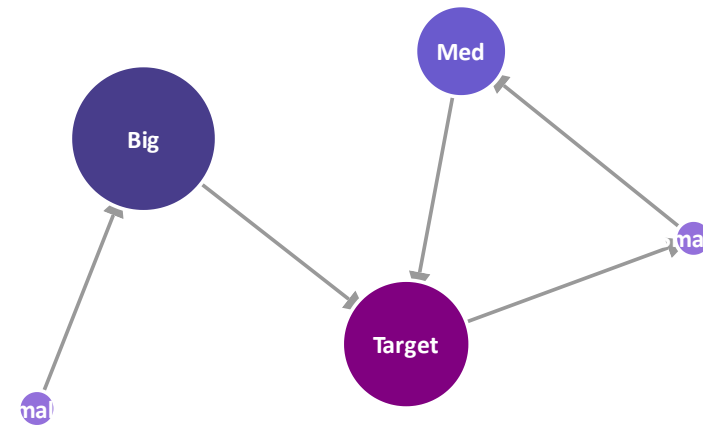
A hyperlink from Page A to Page B is interpreted as a **vote of confidence** or endorsement for Page B.

2. Not All Votes Are Equal

A vote from an **important page** (e.g., Wikipedia, CNN) counts more than a vote from an obscure blog.

3. Recursive Definition

A page is important if it is cited by other important pages. This creates a **recursive** relationship.



Node size represents PageRank score.
Notice how the large node boosts the score of the node it points to.

The Random Surfer Model

PageRank is based on a thought experiment: Imagine a user (a "random surfer") who browses the web indefinitely.



1. Follow Links

Most of the time, the surfer clicks on a hyperlink on the current page.

The choice is **uniform random** among all outgoing links.



2. Teleport (Jump)

Sometimes, the surfer gets "bored" or hits a dead end.

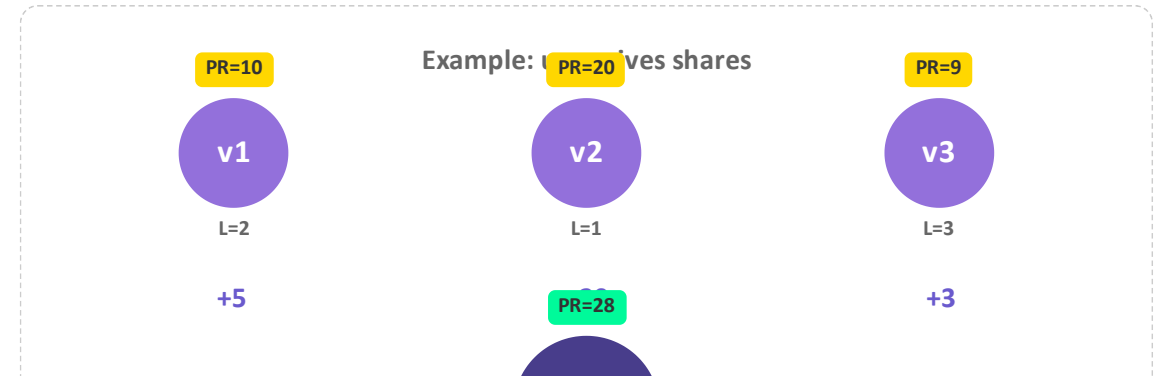
They jump to a **random page** anywhere on the web (typing a new URL).

 **PageRank Definition:** The probability that the random surfer is on page P at any given time step t (in the long run).

The Basic PageRank Formula

$$PR(u) = \sum_{v \in B_u} \frac{PR(v)}{L(v)}$$

- $PR(u)$ PageRank score of page u
- B_u Set of pages that link **to** u (Backlinks)
- $PR(v)$ PageRank score of source page v
- $L(v)$ Number of **outgoing links** from page v



Dead Ends & Spider Traps

The basic PageRank model fails in two common scenarios found on the real web.

💀 Dead Ends

Definition: Pages with no outgoing links.

Consequence: The random surfer gets stuck. Importance "leaks out" of the system, causing total PageRank to converge to 0.



Stuck!

🕷️ Spider Traps

Definition: A group of pages that link only to each other (a cycle).

Consequence: The surfer gets trapped. Importance accumulates infinitely in the trap (Rank Sink).



Infinite Loop

Teleportation (Damping Factor)

The Fix: To solve spider traps and dead ends, we modify the random surfer model. At each step, the surfer has two choices.

α

FOLLOW LINK

Click a hyperlink on the current page uniformly at random.



OR

$1 - \alpha$

TELEPORT

Jump to **any** page in the web graph uniformly at random.



DAMPING FACTOR

$\alpha \approx 0.85$

INTERPRETATION

5-6 Clicks

RESULT

Connected Graph

The surfer browses for a while, then gets "bored" and starts over.

The Final PageRank Formula

$$PR(u) = \frac{1 - \alpha}{N} + \alpha \sum_{v \in B(u)} \frac{PR(v)}{L(v)}$$

"PageRank of u = Probability of Teleporting + Probability of Linking"

$PR(u)$ The PageRank score of page u .

N Total number of pages in the web graph.

$L(v)$ Number of outbound links from page v .

α The **Damping Factor** (usually 0.85). Probability of following a link.

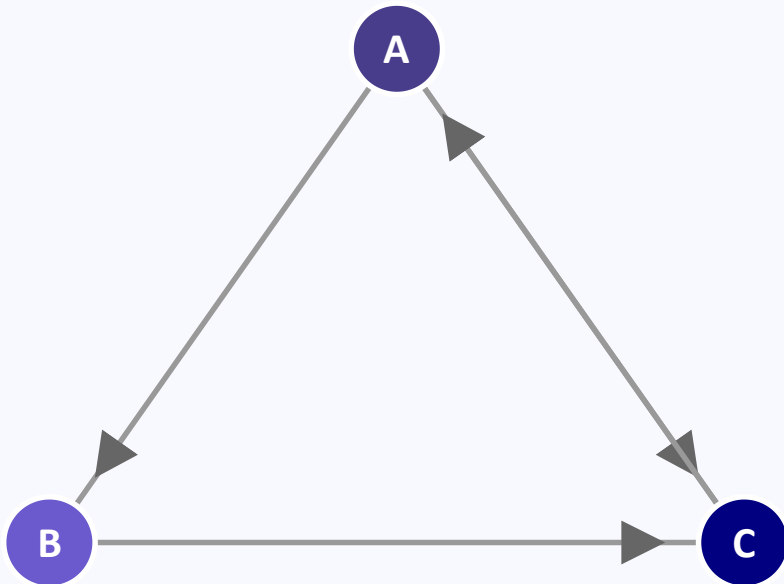
$B(u)$ Set of pages that link **to** u (Backlinks).

$1-\alpha$ Probability of random teleportation (0.15).

Calculation Example: Step 1

Constructing the Stochastic Transition Matrix M

The Web Graph



Transition Matrix M

$$M =$$

	A	B	C	
	0	0	1	A
	1/2	0	0	B
	1/2	1	0	C

Rule: If page j has n outgoing links, then $M_{ij} = 1/n$ for each target i .
(Columns must sum to 1)

Calculation Example: Step 2

First Iteration: Multiplying Transition Matrix by Initial Vector

1. INITIALIZATION (\$V_0\$)

Start with uniform probability distribution:

$$V_0 = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \approx \begin{bmatrix} 0.33 \\ 0.33 \\ 0.33 \end{bmatrix}$$

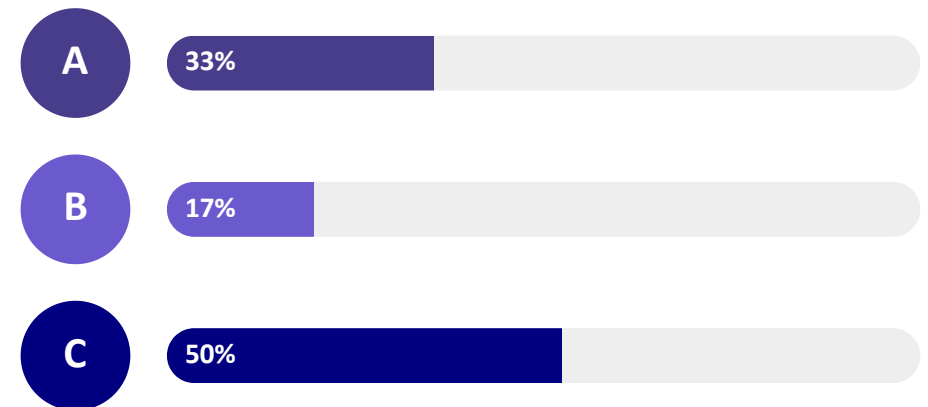
2. ITERATION 1 (\$V_1 = M \times V_0\$)

$$\begin{bmatrix} 0 & 0 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/6 \\ 1/2 \end{bmatrix}$$

Calculation Details:

- \$A = 0 + 0 + 1(1/3) = 0.33\$

Rank Distribution after Iteration 1



Observation:

Node C has the highest rank (0.50) because it receives links from both A and B.

Node B has the lowest (0.17) because it only gets half of A's vote.

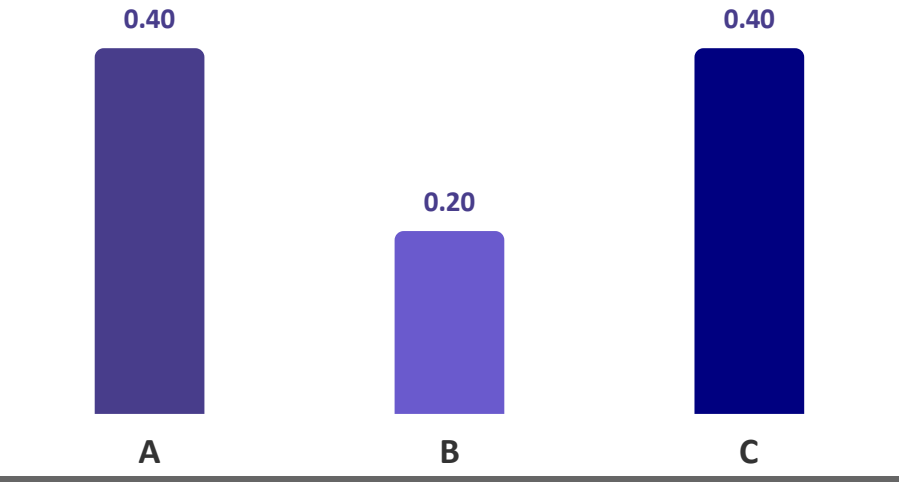
Calculation Example: Step 3

Convergence to Steady State (Power Iteration)

Iteration (\$t\$)	PR(A)	PR(B)	PR(C)
0	0.333	0.333	0.333
1	0.333	0.167	0.500
2	0.500	0.167	0.333
3	0.333	0.250	0.417
...	...	...	...
∞ (Steady)	0.400	0.200	0.400

Key Insight: Regardless of the initial values, the PageRank scores eventually stabilize. This stable distribution is the *Principal Eigenvector* of the matrix M .

Final PageRank Distribution



Nodes A and C are the most important because they point to each other in a cycle, accumulating rank.

Properties of PageRank



Query-Independent

The score is computed **offline**, before any query is issued. It measures the "quality" of a page regardless of content.



Global Measure

It considers the entire structure of the web graph. A link from a distant page can influence the score.



Recursive Definition

A page is important if it is linked to by other important pages. "Votes" are weighted by the voter's importance.



Static Score

The PageRank value changes slowly over time (only when the link graph is updated/recrawled).

CRUCIAL DISTINCTION

PageRank measures **Authority** (Quality), while TF-IDF/BM25 measures **Relevance** (Content Match).
Modern search engines combine both: $\text{Score} = \text{Relevance} \times \text{Authority}$.

Topic-Sensitive PageRank

The Core Idea

Standard PageRank assumes a "random surfer" who gets bored and jumps to **any page** with equal probability ($1/N$).

Topic-Sensitive PageRank biases this jump. The surfer is more likely to jump to pages within a specific **Topic Set** S .

Biased Teleportation

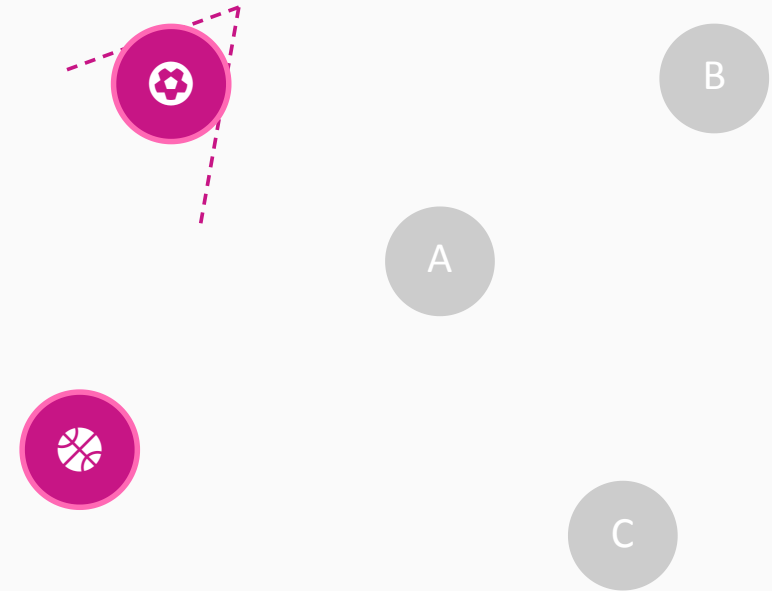
The teleportation vector $\mathbf{v}$ is non-uniform:

$$v_i = \begin{cases} 1/|S| & \text{if } i \in S \\ 0 & \text{if } i \notin S \end{cases}$$

Result: Pages close to the topic set S get higher rank.

Topic: "Sports"

Teleport Only Here!



HITS: Hubs & Authorities

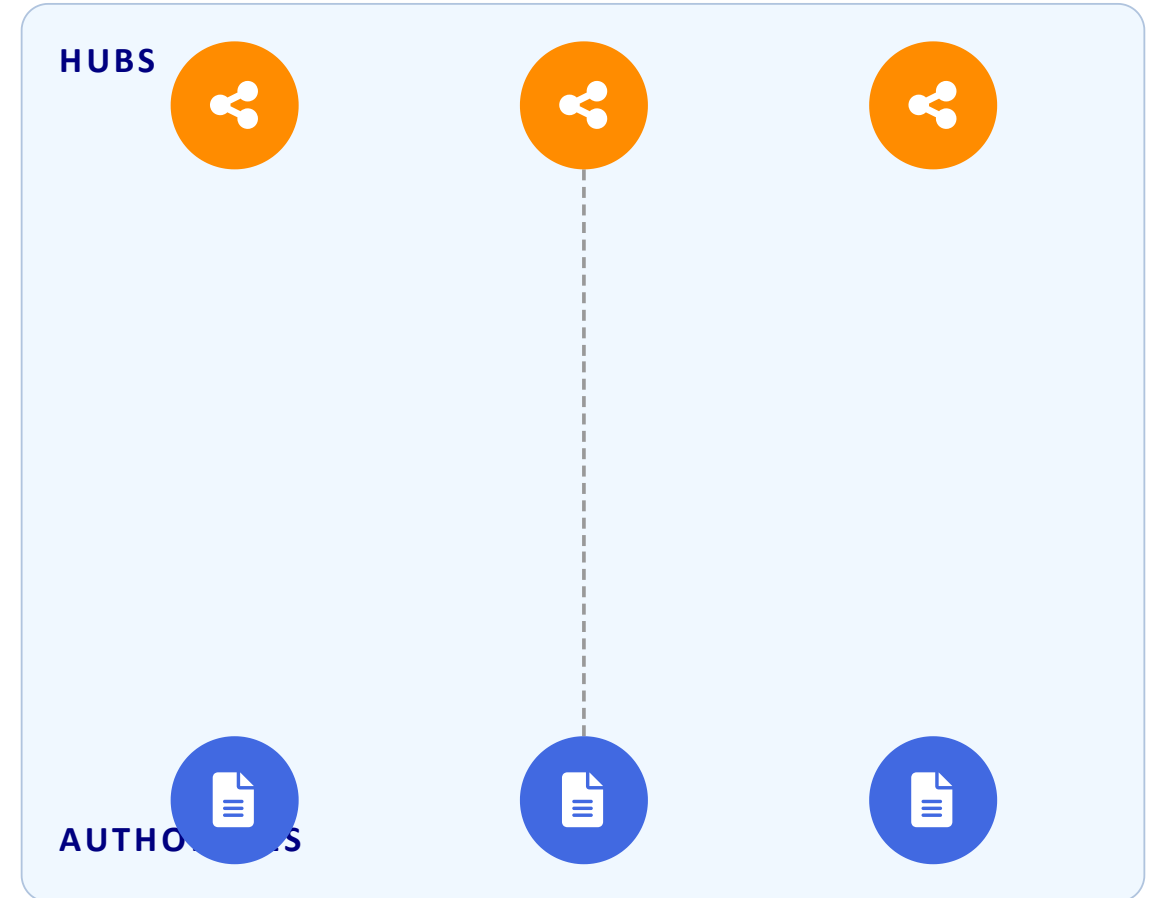
Hyperlink-Induced Topic Search (Jon Kleinberg, 1999)

Query-Dependent Ranking

Unlike PageRank (which is global), HITS is executed **at query time** on a small subset of pages relevant to the query (the "Root Set").

Recursive Definition

- **Hubs:** Pages that link to many good authorities (e.g., lists, directories).
- **Authorities:** Pages that are linked to by many good hubs (e.g., content pages).



Defining Hubs & Authorities



Hubs

Pages that act as **resource lists** or directories. They don't necessarily have the content themselves, but they know where to find it.

(e.g., "Top 10 AI Papers", "Best Hotels in Paris")



Authorities

Pages that contain **valuable, authoritative content** on a specific topic. They are the destination.

(e.g., The actual PDF of the paper, The Hotel's homepage)

Mutually Reinforcing Relationship

Authority Score:

$$a(p) = \sum_{q \rightarrow p} h(q)$$

"Sum of Hub scores pointing to me"



Hub Score:

$$h(p) = \sum_{p \rightarrow q} a(q)$$

"Sum of Authority scores I point to"

*"A good hub points to many good authorities.
A good authority is pointed to by many good hubs."*

The HITS Algorithm

0 Initialization

Set all hub and authority scores to 1:

$$h(p) = 1, \quad a(p) = 1$$

1 Authority Update

Sum scores from hubs pointing to page p :

$$a(p) = \sum_{q \rightarrow p} h(q)$$

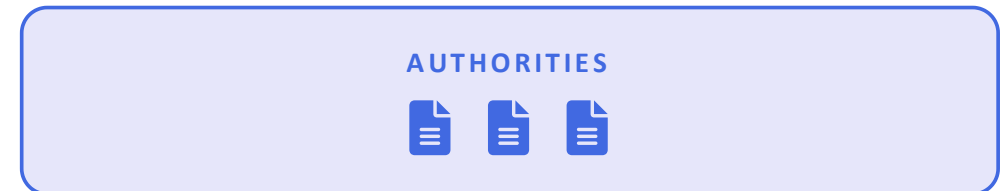
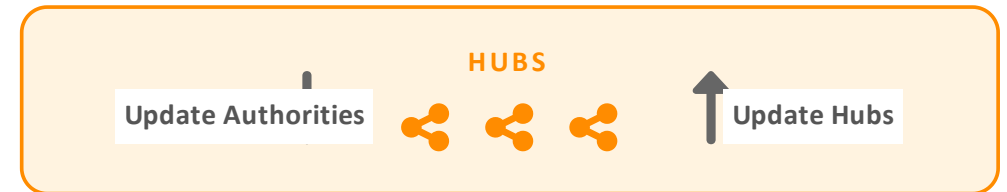
2 Hub Update

Sum scores from authorities pointed to by p :

$$h(p) = \sum_{p \rightarrow q} a(q)$$

3 Normalization

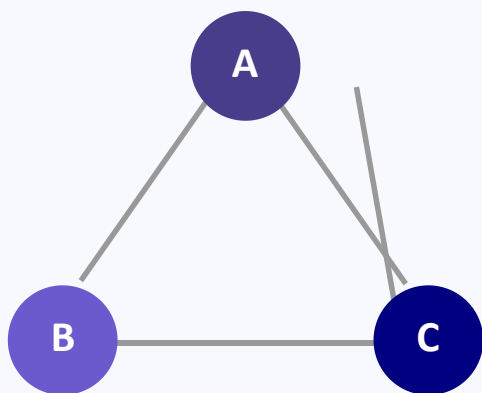
Normalize vectors to unit length (Euclidean) to prevent divergence. Repeat until convergence.



*"I am a good hub if I point to good authorities.
I am a good authority if I am pointed to by good hubs."*

HITS Calculation Example

The Web Graph



Initialization (t=0)

$h = (1, 1, 1)$
 $a = (1, 1, 1)$

1. Update Authorities

$$a(p) = \sum h(q) \text{ for } q \rightarrow p$$

$$a(A) \leftarrow h(C) = 1$$

$$a(B) \leftarrow h(A) = 1$$

$$a(C) \leftarrow h(A) + h(B) = 1 + 1 = 2$$

New $a = (1, 1, 2)$

2. Update Hubs

$$h(p) = \sum a(q) \text{ for } p \rightarrow q$$

$$h(A) \leftarrow a(B) + a(C) = 1 + 2 = 3$$

$$h(B) \leftarrow a(C) = 2$$

$$h(C) \leftarrow a(A) = 1$$

New $h = (3, 2, 1)$

3. Normalization (Euclidean):

Divide vectors by their length ($\sqrt{\sum x^2}$) to keep values small.

$$a_{\text{norm}} \approx (0.41, 0.41, 0.82)$$

$$h_{\text{norm}} \approx (0.80, 0.53, 0.27)$$

PageRank vs. HITS

Comparing the Two Giants

Feature	PageRank (Google)	HITS (Kleinberg)
Scope	Global (Entire Web Graph)	Local (Query-Specific Subgraph)
Query Dependency	Query-Independent Computed offline, same score for all queries.	Query-Dependent Computed at runtime for the specific query.
Model	Random Surfer (Probability)	Hubs & Authorities (Mutual Reinforcement)
Spam Resistance	High (Hard to influence global graph)	Low (Vulnerable to "Topic Drift" and localized spam)
Computation	Expensive offline pre-computation. Fast query time.	Expensive online computation. Slow query time.

Verdict: PageRank won the "Search Engine War" because of its **scalability** and **query-time speed**. HITS is better suited for smaller,

Link Spam & SEO



High Ranking



More Traffic



Revenue



Incentive to Spam

White Hat SEO

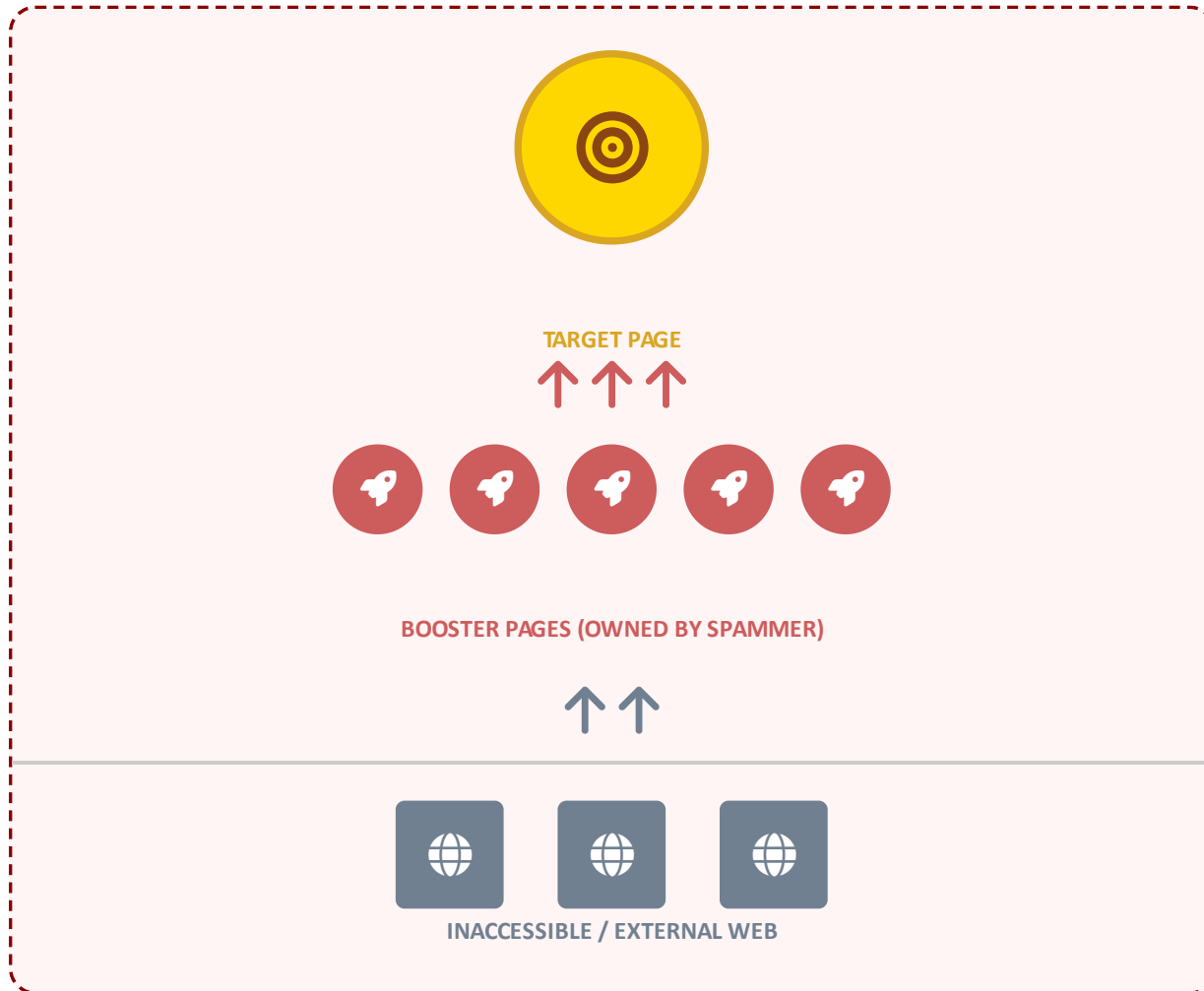
- ✓ Creating high-quality content
- ✓ Improving site speed & UX
- ✓ Proper use of metadata
- ✓ Natural link building

Black Hat SEO (Spam)

- ✗ **Link Farms:** Creating fake sites to link to target
- ✗ **Spam Comments:** Posting links on blogs/wikis
- ✗ **Cloaking:** Showing different content to crawler vs. user
- ✗ **Hidden Text:** Keyword stuffing

SEO = Search Engine Optimization

The Structure of a Spam Farm



⚙️ How it Works

- 1 Create Boosters:** Spammer generates thousands of dummy pages (often with scraped content).
- 2 Funnel Rank:** All booster pages link to the single **Target Page** to concentrate PageRank.
- 3 Recycle Rank:** Target page links back to boosters (or they form a clique) to prevent rank leakage.

Mathematical Intuition:

If a spammer controls M booster pages:

$$PR_{\text{target}} \propto \frac{\alpha \cdot M}{1 - \beta}$$

Rank scales linearly with the number of booster pages (M).

TrustRank: Combating Spam

The Problem

PageRank treats all links as votes. Spammers can easily generate millions of fake links (link farms) to manipulate rankings.

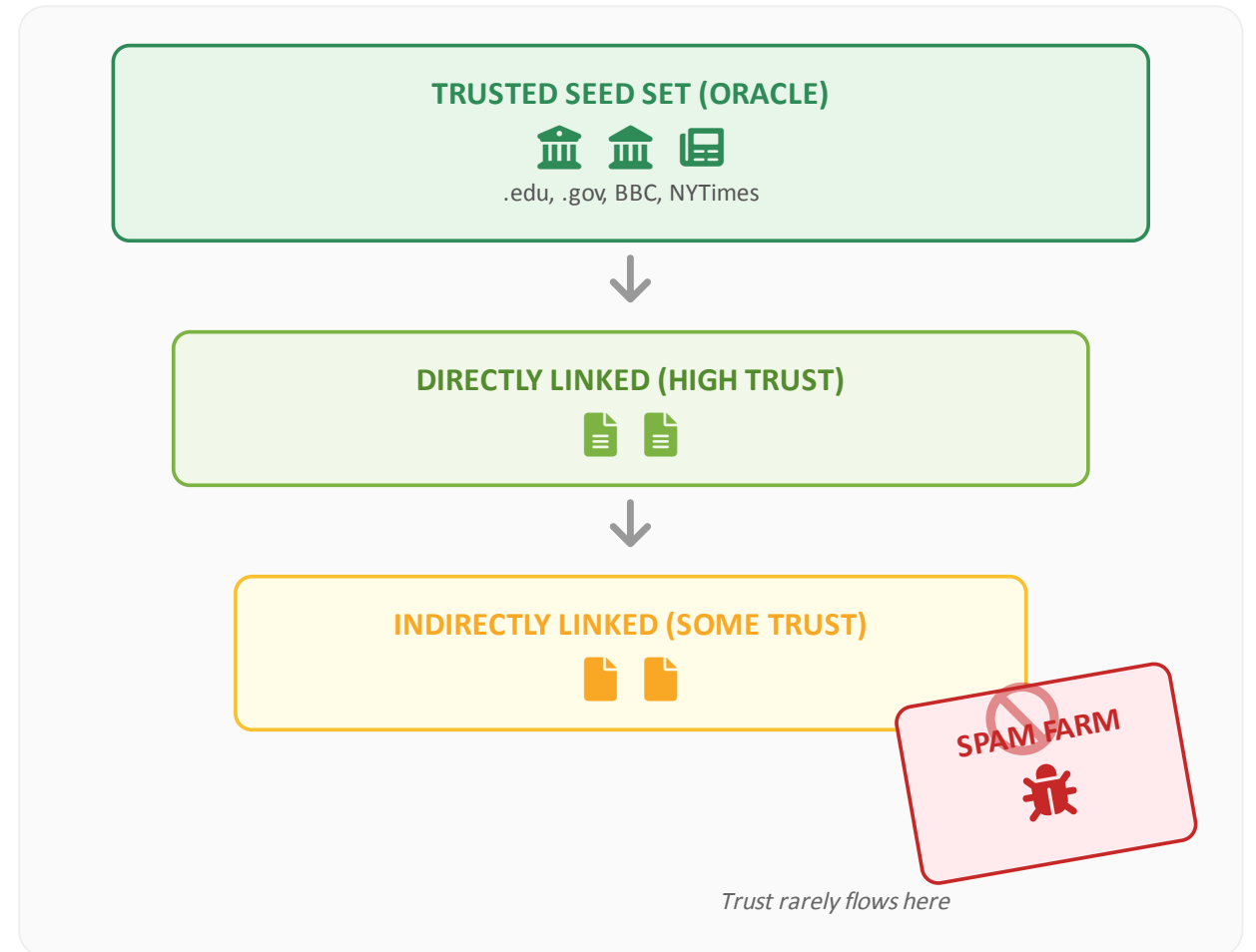
The Solution

We can't manually check the whole web, but we can check a small **Seed Set** of highly reliable pages.

“

"Good pages seldom point to bad pages."

(Approximate Isolation Principle)



TrustRank Algorithm

Biased PageRank

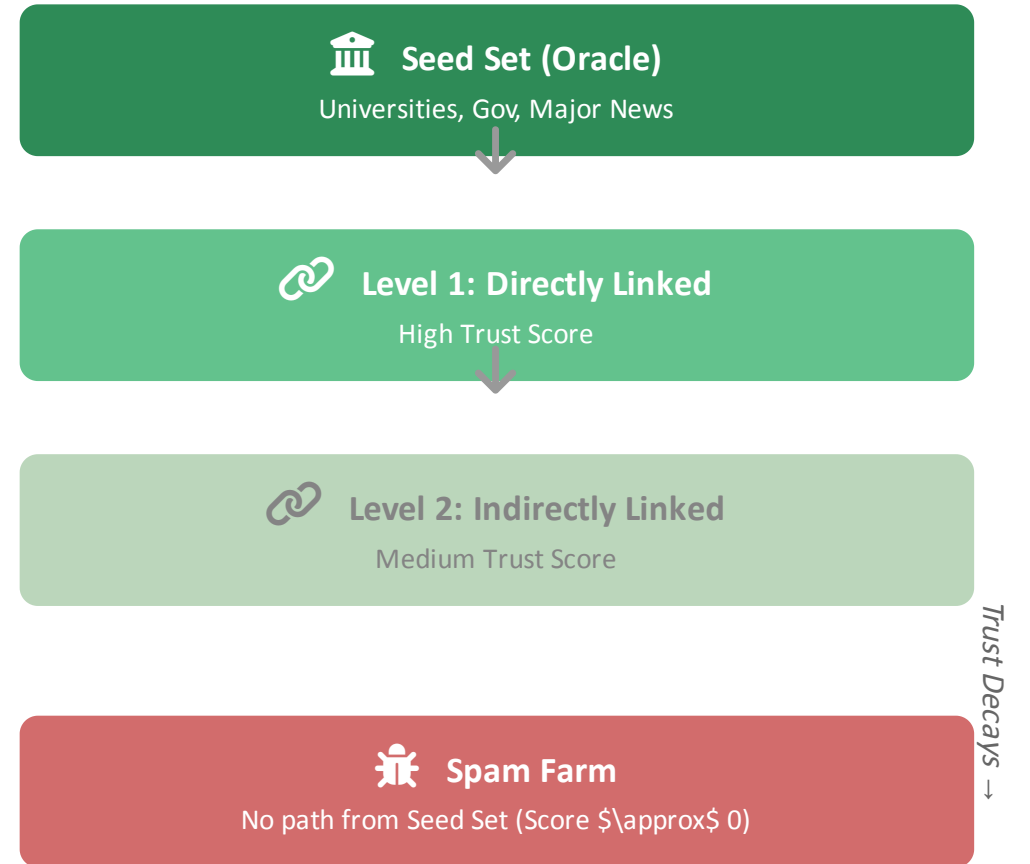
TrustRank is essentially PageRank with a modified **teleportation vector**. Instead of jumping to random pages uniformly, the surfer only jumps to trusted pages.

Standard PageRank Teleport:

$$v_i = \frac{1}{N} \text{ (Uniform)}$$

TrustRank Teleport:

$$v_i = \begin{cases} \frac{1}{|S|} & \text{if } i \in \text{Seed Set } S \\ 0 & \text{otherwise} \end{cases}$$



BadRank & Spam Mass

BadRank

- > **Concept:** The inverse of TrustRank. Instead of propagating trust, we propagate "distrust".
- > **Seed Set:** A set of known spam pages ("Bad Seeds").
- > **Mechanism:** Calculate PageRank using the bad seeds. Pages with high BadRank are likely part of a link farm or spam network.



Detection Strategy

"If you link to spam, or are linked to by spam, you might be spam."

Spam Mass

Quantifies how much of a page's PageRank comes from spam sources versus legitimate sources.

The Formula

$$\text{SpamMass}(p) = \frac{\text{BadRank}(p)}{\text{PageRank}(p)}$$

Spam Probability Scale:



0.0 (Legit)

0.5 (Suspicious)

1.0 (Spam)

Social Network Analysis (SNA)

Graph Theory applied to Society

Investigating social structures through the use of networks and graph theory.



Nodes (Vertices)

People, Organizations, Groups



Edges (Links)

Friendship, Follows, Likes, Citations



Influencer Detection

Identifying key players using centrality measures (Degree, Betweenness, PageRank).



Community Detection

Finding dense clusters of interactions (cliques, echo chambers).



Information Diffusion

Modeling how news, rumors, or viruses spread through the network.

"The Web is essentially a social network where pages are people and links are endorsements."

Centrality Measures in Network Analysis

Beyond simple link counts, centrality measures help identify the most "important" nodes in a graph based on different definitions of importance.

1. Degree Centrality

$$C_D(v)$$

The number of direct connections a node has.

- **Interpretation:** Popularity or local influence.
- **Analogy:** "The Celebrity" - knows many people directly.
- **Web Context:** Number of in-links (In-degree).

2. Closeness Centrality

$$C_C(v)$$

The inverse of the average shortest path distance to all other nodes.

- **Interpretation:** Efficiency of spreading information.
- **Analogy:** "The Broadcaster" - can reach everyone quickly.
- **Web Context:** Fewer clicks to reach any page.

3. Betweenness Centrality

$$C_B(v)$$

The fraction of shortest paths between node pairs that pass through this node.

- **Interpretation:** Control over information flow.
- **Analogy:** "The Bridge" - connects disparate groups.
- **Web Context:** Critical hubs connecting communities.

Relation to PageRank

PageRank is a variant of *Eigenvector Centrality*. It acknowledges that not all connections are equal - a link from a high centrality

Community Detection

Uncovering Structure in Complex Networks

26

What is a Community?

A group of nodes with **dense internal connections** and **sparse external connections** to the rest of the network.

The "Bridge" Insight

Edges connecting different communities act as "bridges". These edges have high **Edge Betweenness** because many shortest paths must pass through them.

 High Betweenness = Weak Tie

1

Calculate Betweenness

Compute edge betweenness for all edges in the graph.

2

Remove Edge

Find and remove the edge with the **highest** betweenness score.

3

Recalculate

Betweenness scores change after removal. Re-compute for remaining edges.

4

Repeat

Continue until no edges remain or desired clusters are formed.

Modern Link Analysis

Beyond the Static Hyperlink: Social, Apps, and Entities

1. Social Media Signals

- **"Soft Links"**: Likes, Retweets, and Shares act as endorsements, similar to hyperlinks.
- **Velocity**: Links accumulate slowly; social signals explode rapidly (Virality).
- **Identity**: We know *who* endorsed it (User Authority), unlike anonymous web links.

2. User Interaction (Implicit)

- **Clickstreams**: Actual usage paths > theoretical link paths.
- **Dwell Time**: Did the user stay? (Quality signal).
- **Pogo-sticking**: Did they return to search immediately?

3. The "Walled Garden" Problem

Much of the modern web is hidden inside mobile apps (Instagram, TikTok) where traditional crawlers cannot go.

Challenge:

Link analysis breaks when content is isolated in silos without HTTP links.

4. Knowledge Graphs

- **Entities, not Pages**: Linking "Tom Cruise" to "Top Gun" (Relationship) rather than Page A to Page B.
- **Google Knowledge Graph**: Uses link analysis on *facts* to

Lab: Implementing PageRank

Calculating scores using Python and NetworkX

Steps

1. Import the **NetworkX** library.
2. Create a **Directed Graph** (DiGraph).
3. Add edges to represent hyperlinks (Source → Target).
4. Run pagerank with damping factor **alpha**.

Key Parameter

alpha=0.85: damping factor (probability of following links vs teleporting).

```
pagerank_demo.py

import networkx as nx

# 1. Create directed graph
G = nx.DiGraph()

# 2. Add edges
G.add_edges_from([('A', 'B'), ('A', 'C'), ('B', 'C'), ('C', 'A'), ('D', 'C')])

# 3. PageRank (alpha = 0.85)
pr = nx.pagerank(G, alpha=0.85)

# 4. Print sorted results
for n,s in sorted(pr.items(), key=lambda x: x[1], reverse=True):
    print(f"{n}: {s:.4f}")
```


Lab: Implementing HITS Algorithm

The HITS algorithm iteratively updates **Hub** and **Authority** scores. We use matrix operations for efficiency and normalize scores to prevent numerical overflow.

```
import numpy as np

def hits_algorithm(adj_matrix, max_iter=100, tol=1e-6):
    n = len(adj_matrix)
    hubs = np.ones(n)
    auths = np.ones(n)

    for _ in range(max_iter):
        # Update Authorities:  $A = M^T * H$ 
        prev_auths = auths.copy()
        auths = np.dot(adj_matrix.T, hubs)
        auths /= np.linalg.norm(auths) # Normalize
```

Key Insight: Unlike PageRank, HITS requires two vectors (hubs and authorities) and uses the adjacency matrix and its transpose for updates.

Case Study: Google's Evolution

From Pure Link Analysis to AI-Driven Understanding

1998 - 2010

The Link Era

- **PageRank Core:** Links were the primary signal for authority.
- **Anchor Text:** Heavily relied upon for relevance.
- **Vulnerability:** Susceptible to link farms and buying links (Google Bombing).

2010 - 2015

Semantic & Clean-Up

- **Penguin Update (2012):** Penalized unnatural link profiles and spam.
- **Knowledge Graph:** Shift from "strings" to "things" (Entities).
- **TrustRank:** Focus on seed sets of trusted sites.

2015 - Present

The AI Era

- **RankBrain:** Machine learning to interpret ambiguous queries.
- **BERT & MUM:** Deep natural language understanding.
- **Role of Links:** Still important, but now one of hundreds of signals (User Behavior, Content Quality).

e sc

criti

Summary: Link Analysis

Key Takeaways from Week 7

01 PageRank (The Foundation)

Links are votes. The **Random Surfer Model** with teleportation calculates global importance, handling dead ends and spider traps.

02 HITS (Hubs & Authorities)

A query-dependent approach identifying **Authorities** (good content) and **Hubs** (good links). Vulnerable to topic drift.

03 The Spam War

Link farms and honey pots try to manipulate rank. Search engines evolved with **TrustRank**, **Penguin**, and AI to detect unnatural patterns.

04 Personalization

Modern search uses **Topic-Sensitive PageRank** and user history to tailor results, moving beyond a "one size fits all" ranking.

Recommended Reading

Deep Dive into Link Analysis

Textbook

- **Introduction to Information Retrieval** (Manning, Raghavan, Schütze)
 - Chapter 21: *Link Analysis*

Seminal Papers

- **Brin & Page (1998): "The Anatomy of a Large-Scale Hypertextual Web Search Engine"**
The original Google paper introducing PageRank.
- **Kleinberg (1999): "Authoritative Sources in a Hyperlinked Environment"**
The foundation of the HITS algorithm.
- **Haveliwala (2002): "Topic-Sensitive PageRank"**
Personalizing search results based on user interests.



Questions?

Thank you for your attention.

NEXT WEEK:

Evaluation in Information Retrieval

Precision, Recall, MAP, and User Studies

PageRank Iterations

Step-by-Step Calculation

The Graph



Assumption: $\beta = 0.8$ (Teleport probability = 0.2).
Dangling node B distributes rank equally to all.

Iteration	PR(A)	PR(B)	PR(C)
0 (Init)	0.333	0.333	0.333
1	0.373	0.313	0.313
2	0.395	0.302	0.302
3	0.408	0.296	0.296
Converged	0.421	0.289	0.289

The Matrix View

Linear Algebra of the Web

$$\begin{bmatrix} \text{PR}(A)' \\ \text{PR}(B)' \\ \text{PR}(C)' \end{bmatrix} = \begin{bmatrix} 0 & 1/2 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1/2 & 0 \end{bmatrix} \times \begin{bmatrix} \text{PR}(A) \\ \text{PR}(B) \\ \text{PR}(C) \end{bmatrix}$$

New Rank Vector

Distribution after 1 step

Transition Matrix M

Probabilities of moving from j to i.
Columns sum to 1.

Current Rank Vector

Distribution at current step

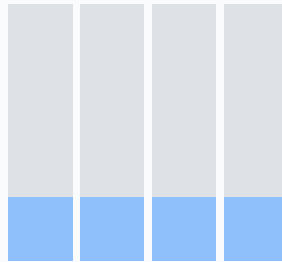
$$v_{\text{new}} = M \times v_{\text{old}}$$

Personalized PageRank

Biasing the Random Surfer

Standard PageRank

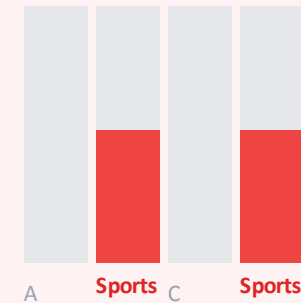
Teleport to ANY page with equal probability.



$$\mathbf{v} = [1/N, 1/N, 1/N, 1/N]$$

Topic-Sensitive PageRank

Teleport **ONLY** to pages in the **Topic Set**.



$$\mathbf{v} = [0, 0.5, 0, 0.5]$$

Result: Rank flows preferentially to Sports pages and their neighbors.

Web Spam: The Dark Side

Manipulating Search Engines for Profit

T Term Spam

Manipulating the **text content** of a page to appear relevant.

- **Keyword Stuffing:** Repeating keywords hundreds of times ("cheap loans cheap loans...").
- **Hidden Text:** White text on white background.
- **Cloaking:** Showing different content to the crawler vs. the user.

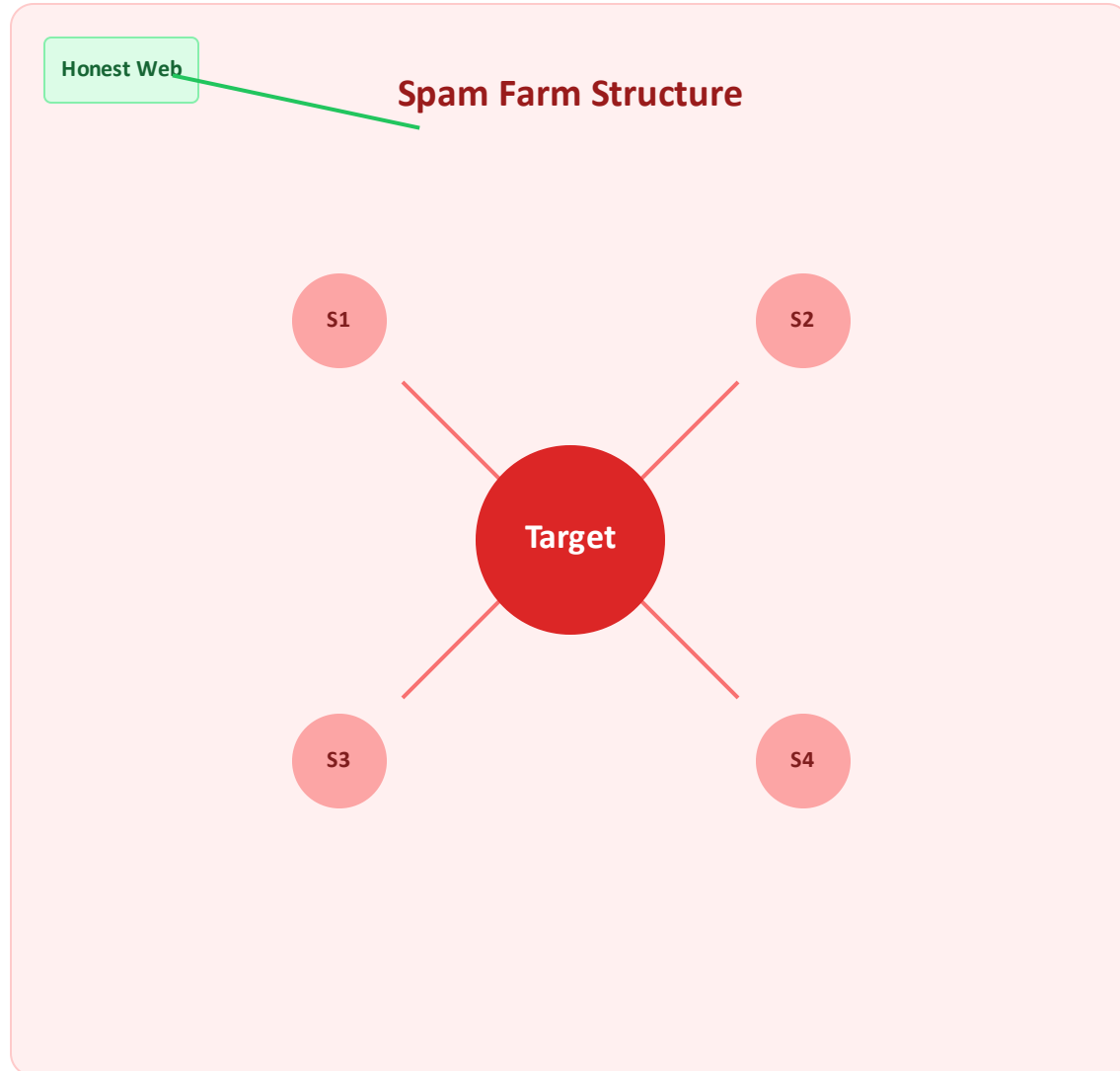
L Link Spam

Manipulating the **link structure** to boost PageRank/Authority.

- **Link Farms:** Networks of sites linking to each other.
- **Spam Blogs (Splogs):** Fake blogs created solely for links.
- **Comment Spam:** Posting links in forums/blogs ("Nice post! Visit my casino...").

Collusion & Honey Pots

Anatomy of a Spam Farm



The Goal

Boost the PageRank of the **Target Page** by funneling link authority from thousands of dummy pages.

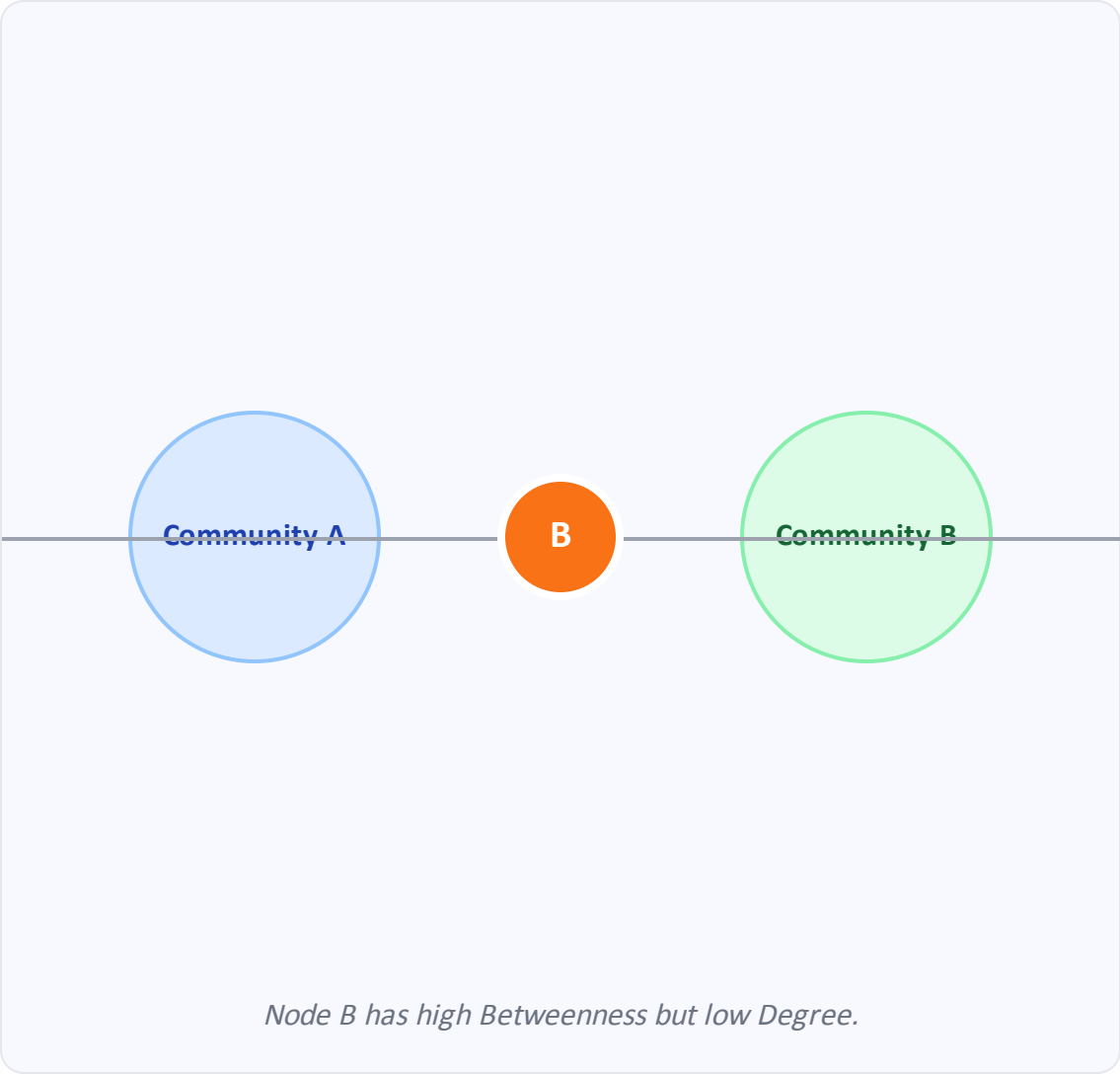
Components:

- **Target Page:** The page intended to rank high.
- **Supporter Pages:** Millions of generated pages linking to the target (Link Farm).
- **Honey Pots:** Useful pages (e.g., Linux guides) created to attract legitimate links from the honest web.

Defense: Algorithms like **TrustRank** identify spam farms by starting from trusted seeds (e.g., .edu, .gov) and penalizing distant nodes.

Betweenness Centrality

Identifying Bridges & Gatekeepers



Definition

Measures the number of times a node acts as a bridge along the **shortest path** between two other nodes.

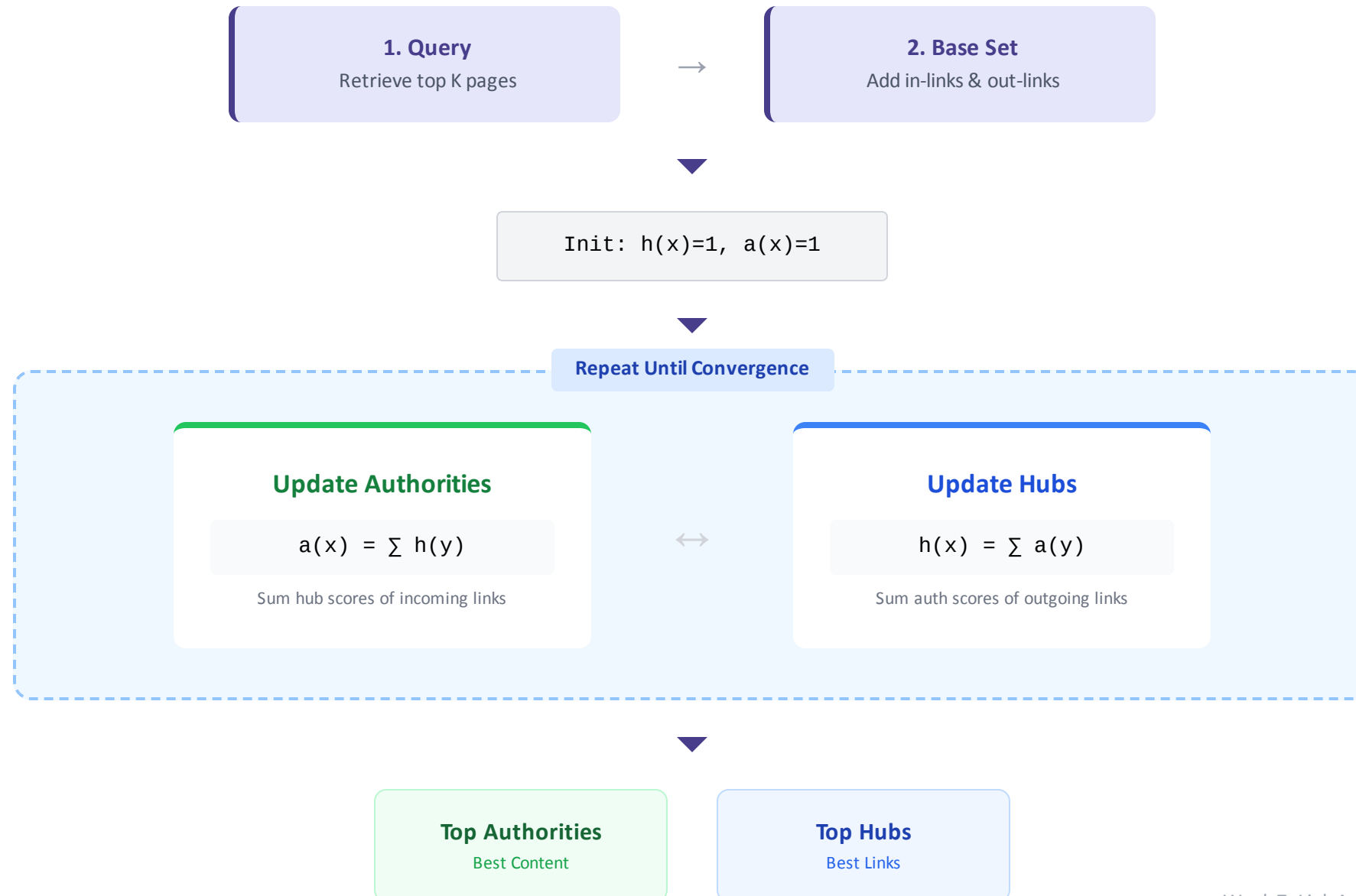
Why It Matters:

- **Information Flow:** Controls the spread of information between groups.
- **Vulnerability:** Removing high-betweenness nodes can fragment the network.
- **Applications:** Identifying key influencers, traffic bottlenecks, or biological pathways.

Metric	Focus
Degree Centrality	Popularity (Number of connections)
Betweenness	Control (Bridge between groups)
Closeness	Efficiency (Distance to others)

HITS Algorithm Flowchart

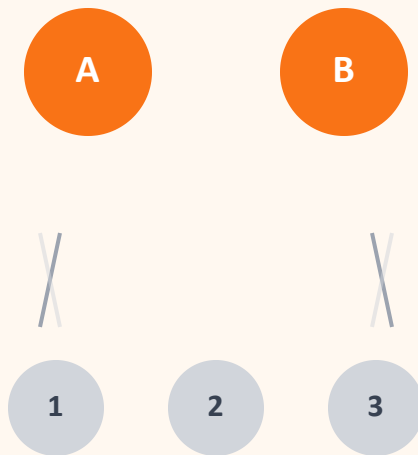
Iterative Calculation of Hubs & Authorities



SimRank: Structural Similarity

"Two objects are similar if they are related to similar objects"

Recursive Definition



A and B are similar because they share neighbors (1, 2, 3).

The Intuition

Similarity flows through the graph. Just as PageRank flows authority, SimRank flows similarity.

(Analogy: Two professors are similar if they advise the same students.)

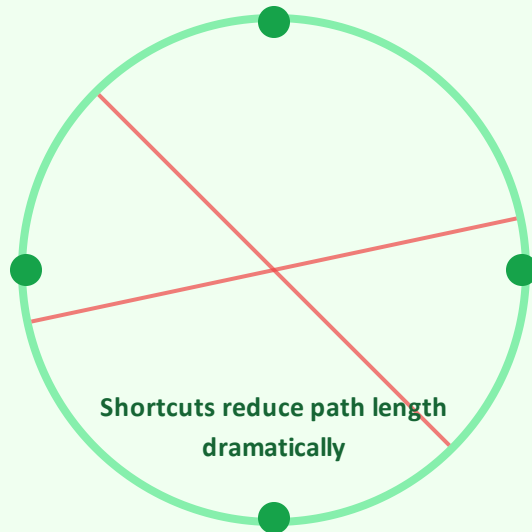
Applications:

- **Recommender Systems:** "People who bought X also bought Y".
- **Social Networks:** Friend suggestions based on mutual friends.
- **Citation Analysis:** Papers citing similar sources are likely related.

$$s(a,b) = (C / |N(a)||N(b)|) \sum s(i,j)$$

Small World Phenomenon

Six Degrees of Separation



The Concept

Despite billions of pages, any two pages on the Web are typically only **19 clicks away** from each other.

Key Properties:

- **High Clustering:** "My friends know each other." (Local structure)
- **Short Path Length:** "I can reach anyone quickly." (Global efficiency)
- **Hubs:** Highly connected nodes (e.g., Wikipedia, Google) act as shortcuts.

Implication for Crawling: Search engines can discover the entire web quickly by following links, thanks to the small diameter.

Link vs. Content Analysis

Combining Relevance and Authority

C Content Analysis

- ✓ **Focus:** What the page *says* (Keywords, TF-IDF, Structure).
- ✓ **Goal:** Determine **Relevance** to the user's query.
- ✗ **Weakness:** Vulnerable to "Keyword Stuffing" and hidden text.

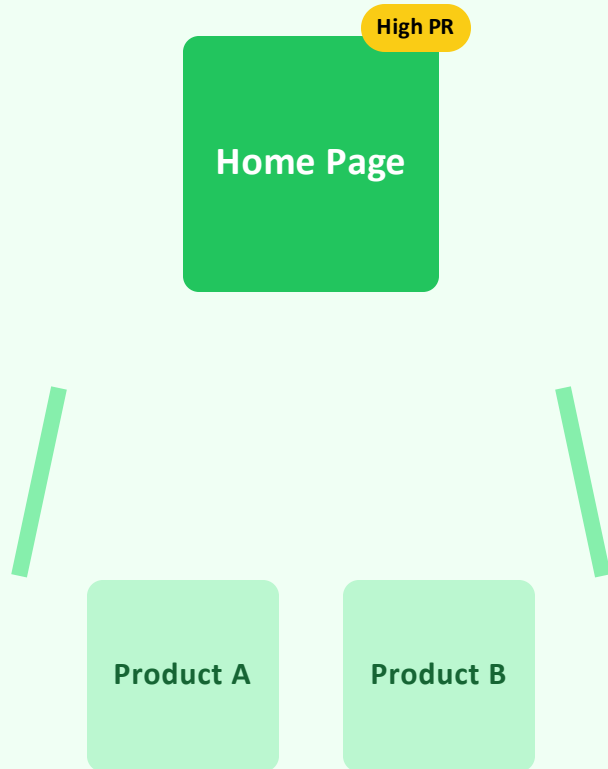
L Link Analysis

- ✓ **Focus:** What others *say* about the page (Backlinks, Anchor Text).
- ✓ **Goal:** Determine **Authority** and **Quality**.
- ✗ **Weakness:** Vulnerable to "Link Farms" and paid links.

Modern Search Engines = Content (Relevance) × Links (Authority)

Link Juice & SEO

Managing Authority Flow



Authority flows from the Home Page to sub-pages.

The Concept

"Link Juice" is a colloquial term for the **PageRank** or authority passed from one page to another via hyperlinks.

Key Principles:

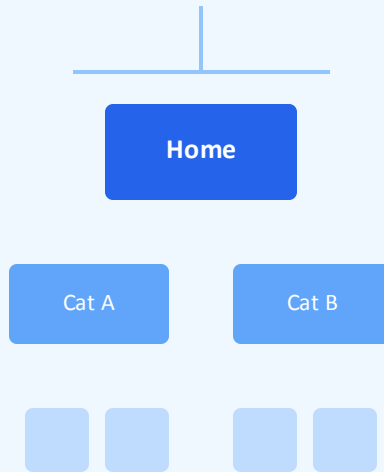
- **Flow:** Juice flows through standard `<a href>` links.
- **Dilution:** More outgoing links on a page = less juice passed per link.
- **Internal Linking:** Crucial for distributing authority to deep pages.

Leakage: Linking to external sites passes authority away. Use `rel="nofollow"` to prevent this.

Internal Linking Strategies

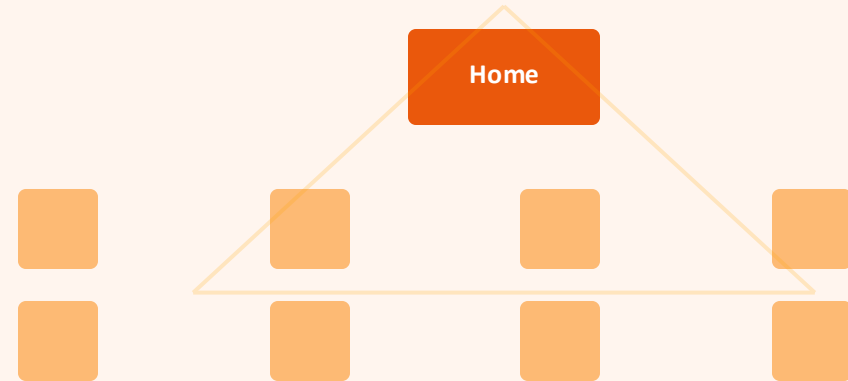
Siloing vs. Flat Architecture

The Silo Structure



Pros: Concentrates topical authority. Helps search engines understand site structure.

Flat Structure

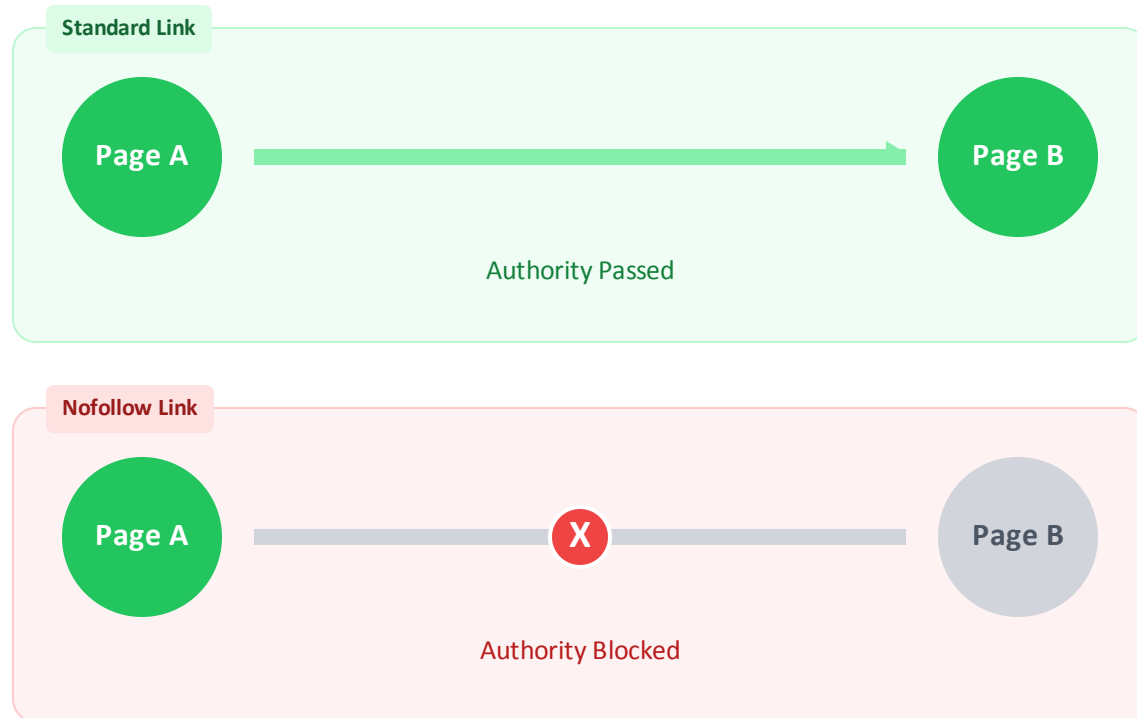


Pros: All pages near home.

Cons: Dilutes topical relevance. Harder to rank for specific keywords.

The Nofollow Attribute

Controlling Link Authority Flow



Syntax

```
<a href="http://example.com" rel="nofollow">Link Text</a>
```

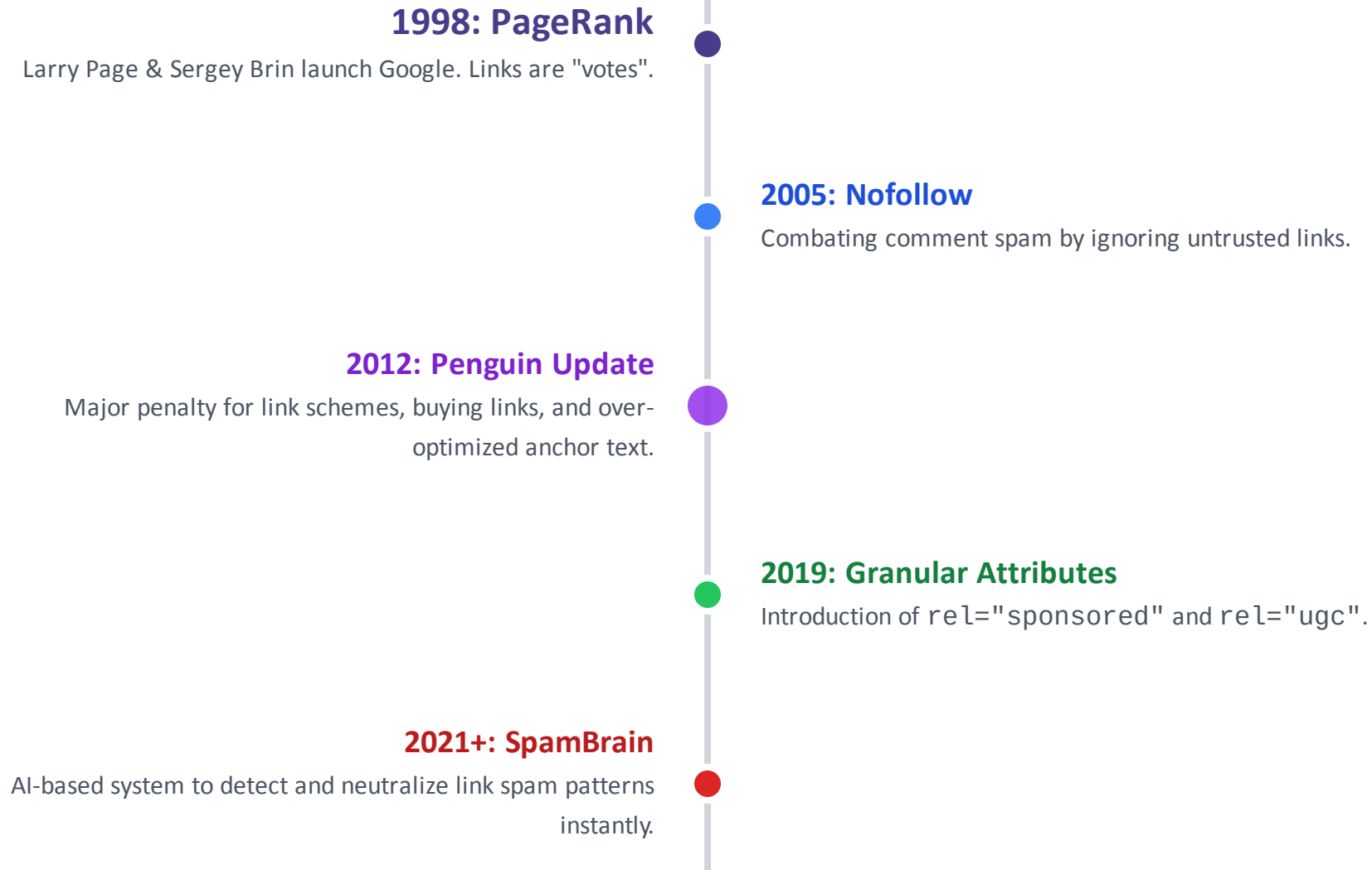
When to Use:

- **Untrusted Content:** Blog comments, forum posts (prevents spam).
- **Paid Links:** Advertisements or sponsored content (avoids penalties).
- **Crawl Prioritization:** Prevent bots from wasting time on login pages or filters.

Evolution (2019): Google introduced `rel="sponsored"` for ads and `rel="ugc"` for user-generated content as granular options.

Evolution of Link Analysis

From Quantity to Quality

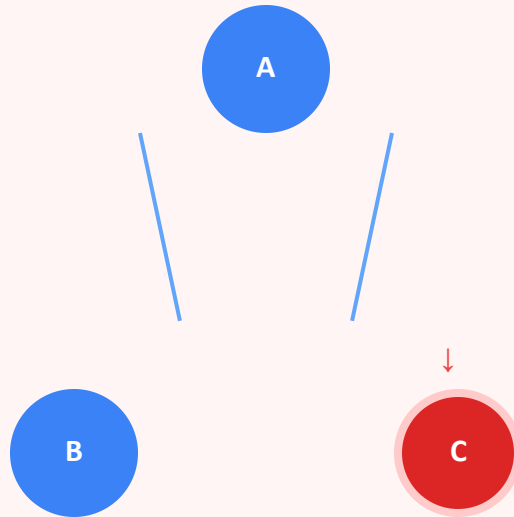


Handling Dangling Nodes

Solving the "Rank Sink" Problem

The Problem

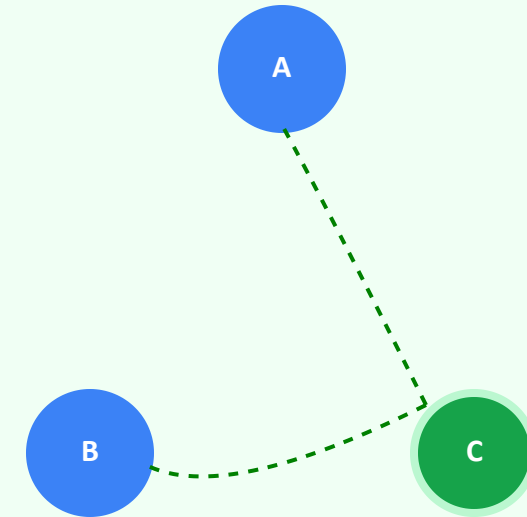
Rank flows in but never leaves.



Result: Total PageRank decreases in each iteration ($\text{Sum} < 1$).

The Solution

Add virtual links to everywhere.



Fix: Treat dangling nodes as if they link to *all* pages with probability $1/N$.

Computing PageRank: Power Iteration

Finding the Steady State Vector

1. Initialize

$$\mathbf{x}^{(0)} = [1/N, \dots, 1/N]^T$$

2. Multiply

$$\mathbf{x}^{(k+1)} = \mathbf{M} \cdot \mathbf{x}^{(k)}$$

3. Check Convergence

$$|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}| < \epsilon$$

Why It Works

Since the transition matrix **M** is stochastic and irreducible (thanks to teleportation), the sequence of vectors is guaranteed to converge to the unique **Principal Eigenvector**.

Efficiency:

- **Scalable:** We don't need to invert the matrix.
- **Sparse Matrix:** M is mostly zeros (sparse), making multiplication fast.
- **Iterations:** Typically converges in ~50-100 iterations for web-scale graphs.

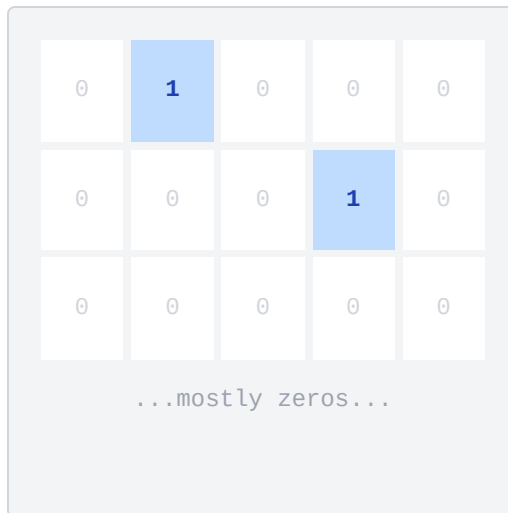
Optimization: Sparse Matrices

Handling Web-Scale Data

The Problem: Dense Matrix

Storing all $N \times N$ entries is impossible.

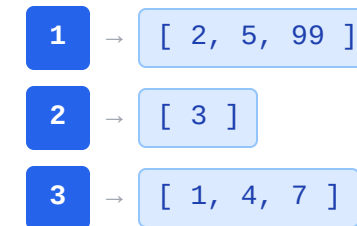
$N = 50 \text{ Billion} \rightarrow N^2 = 2.5 \times 10^{21}$ (Zettabytes)



The Solution: Sparse List

Store only non-zero links (Edges).

Size $\approx |E|$ (Number of Links)



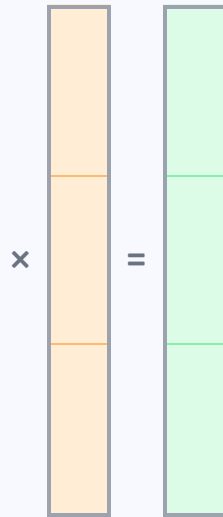
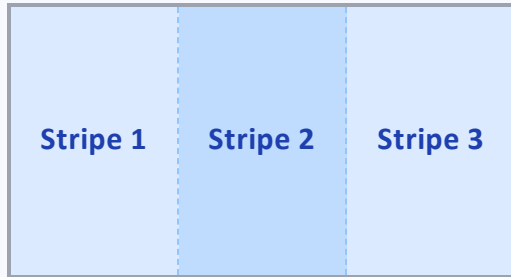
MapReduce Implementation:

- Map: Emit (target, source_rank / out_degree)
- Reduce: Sum all incoming rank contributions.

Handling Scale: Block-Stripe Update

Processing Graphs Larger than RAM

Matrix M (Striped)



Vector x is also split into blocks.

The Constraint

For billions of pages, even the rank vector x (8 bytes * 50B pages = 400GB) may exceed RAM capacity of a single machine.

The Strategy:

- **Partition:** Break the vector x into k blocks.
- **Stream:** Break the matrix M into corresponding vertical stripes.
- **Compute:** Load one block of x and stream the matching stripe of M to compute a portion of x_{new} .

Cost: Limited by Disk I/O speed, not CPU. Efficient sequential reads are key.

Personalized Search

Beyond Global PageRank



The Traveler

History: Flights, Hotels, Bali

Query: "Java"

[Java Travel Guide - Indonesia](#)

www.lonelyplanet.com/indonesia/java

Explore the volcanoes and temples of Java...

[Best Coffee in Java](#)

www.coffeereview.com/java

Top rated beans from the island...



The Developer

History: GitHub, StackOverflow

Query: "Java"

[Download Java for Windows](#)

www.java.com/download

Get the latest JRE version...

[Java Tutorial - W3Schools](#)

www.w3schools.com/java

Learn Java programming language...

The Mechanism:

Modify the **Teleport Vector (v)** in PageRank.

$$PR(p) = (1-\alpha)M \cdot PR + \alpha \mathbf{v}_{user}$$

PageRank vs. HITS

Comparing the Two Giants

Feature	PageRank (Google)	HITS (Kleinberg)
Scope	Global (Entire Web Graph)	Local (Query-Specific Subgraph)
Query Dependency	Query-Independent Computed offline, same score for all queries.	Query-Dependent Computed at runtime for the specific query.
Model	Random Surfer (Probability)	Hubs & Authorities (Mutual Reinforcement)
Spam Resistance	High (Hard to influence global graph)	Low (Vulnerable to "Topic Drift" and localized spam)
Computation	Expensive offline pre-computation. Fast query time.	Expensive online computation. Slow query time.

Verdict: PageRank won the "Search Engine War" because of its **scalability** and **query-time speed**. HITS is better suited for smaller,

Summary: Link Analysis

Key Takeaways from Week 7

01 PageRank (The Foundation)

Links are votes. The **Random Surfer Model** with teleportation calculates global importance, handling dead ends and spider traps.

02 HITS (Hubs & Authorities)

A query-dependent approach identifying **Authorities** (good content) and **Hubs** (good links). Vulnerable to topic drift.

03 The Spam War

Link farms and honey pots try to manipulate rank. Search engines evolved with **TrustRank**, **Penguin**, and AI to detect unnatural patterns.

04 Personalization

Modern search uses **Topic-Sensitive PageRank** and user history to tailor results, moving beyond a "one size fits all" ranking.

Recommended Reading

Deep Dive into Link Analysis

Textbook

- **Introduction to Information Retrieval** (Manning, Raghavan, Schütze)
 - Chapter 21: *Link Analysis*

Seminal Papers

- **Brin & Page (1998): "The Anatomy of a Large-Scale Hypertextual Web Search Engine"**
The original Google paper introducing PageRank.
- **Kleinberg (1999): "Authoritative Sources in a Hyperlinked Environment"**
The foundation of the HITS algorithm.
- **Haveliwala (2002): "Topic-Sensitive PageRank"**
Personalizing search results based on user interests.



Questions?

Thank you for your attention.

NEXT WEEK:

Evaluation in Information Retrieval

Precision, Recall, MAP, and User Studies

PageRank Iterations

Step-by-Step Calculation

The Graph



Assumption: $\beta = 0.8$ (Teleport probability = 0.2).
Dangling node B distributes rank equally to all.

Iteration	PR(A)	PR(B)	PR(C)
0 (Init)	0.333	0.333	0.333
1	0.373	0.313	0.313
2	0.395	0.302	0.302
3	0.408	0.296	0.296
Converged	0.421	0.289	0.289

The Matrix View

Linear Algebra of the Web

$$\begin{bmatrix} \text{PR}(A)' \\ \text{PR}(B)' \\ \text{PR}(C)' \end{bmatrix} = \begin{bmatrix} 0 & 1/2 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1/2 & 0 \end{bmatrix} \times \begin{bmatrix} \text{PR}(A) \\ \text{PR}(B) \\ \text{PR}(C) \end{bmatrix}$$

New Rank Vector

Distribution after 1 step

Transition Matrix M

Probabilities of moving from j to i.
Columns sum to 1.

Current Rank Vector

Distribution at current step

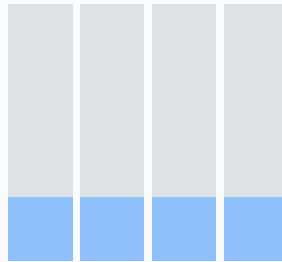
$$v_{\text{new}} = M \times v_{\text{old}}$$

Personalized PageRank

Biasing the Random Surfer

Standard PageRank

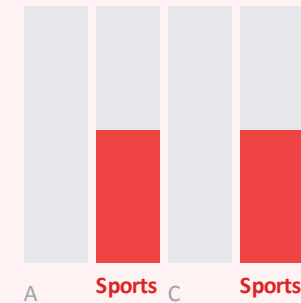
Teleport to ANY page with equal probability.



$$\mathbf{v} = [1/N, 1/N, 1/N, 1/N]$$

Topic-Sensitive PageRank

Teleport **ONLY** to pages in the **Topic Set**.



$$\mathbf{v} = [0, 0.5, 0, 0.5]$$

Result: Rank flows preferentially to Sports pages and their neighbors.

Web Spam: The Dark Side

Manipulating Search Engines for Profit

Term Spam

Manipulating the **text content** of a page to appear relevant.

- **Keyword Stuffing:** Repeating keywords hundreds of times ("cheap loans cheap loans...").
- **Hidden Text:** White text on white background.
- **Cloaking:** Showing different content to the crawler vs. the user.

Link Spam

Manipulating the **link structure** to boost PageRank/Authority.

- **Link Farms:** Networks of sites linking to each other.
- **Spam Blogs (Splogs):** Fake blogs created solely for links.
- **Comment Spam:** Posting links in forums/blogs ("Nice post! Visit my casino...").